

JAVA ASSIGNMENT-1

SINGLE LEVEL INHERITANCE

1. Basic Information

Experiment No.: 01

Date: 17/09/2026

Title: Implementation of Single-Level Inheritance in Java

2. Objective / Aim

To design and implement a Java program to demonstrate **Single-Level Inheritance**, where one subclass inherits the properties and methods of a single superclass.

3. System Requirements

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Hard Disk: Minimum 10 GB free space
- Keyboard and Monitor

Software Requirements

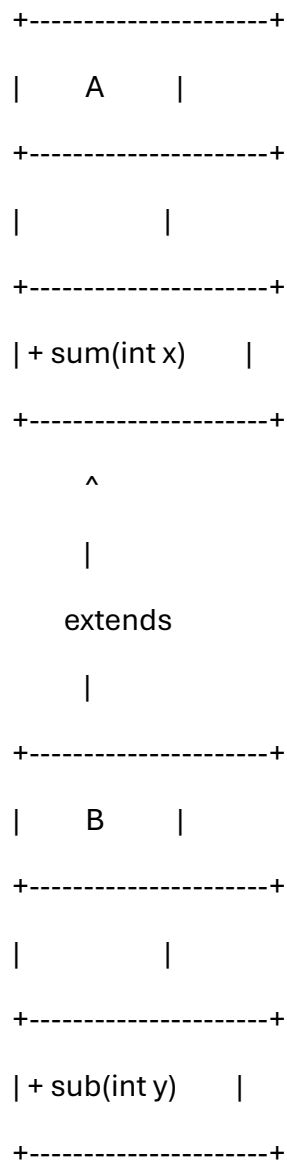
- Operating System: Windows/Linux/macOS
- JDK: JDK 8 or above
- IDE: IntelliJ IDEA, Eclipse, NetBeans, or Visual Studio Code
- Command Prompt/Terminal

4. Algorithm / Logic Description

1. Start the program.
2. Create a superclass A.
3. Define the sum() method in class A.
4. Create a subclass B using the extends keyword.
5. The class B inherits the sum() method from class A.
6. Define the sub() method in class B.
7. Create an object obj of class B.

8. Call the inherited sum() method using the object.
9. Call the sub() method using the same object.
10. Display the values.
11. Stop the program.

5. UML Class Diagram



6. Source Code

```

class A{
    void sum(int x){
        System.out.println("the value of x is :"+x);
    }
}
  
```

```
}  
  
public class B extends A{  
  
    void sub(int y){  
        System.out.println("the value of y is :"+(y));  
    }  
  
    public static void main(String[] args) {  
        B obj = new B();  
        obj.sum(10);  
        obj.sub(20);  
    }  
}
```

7. Compilation & Execution Commands

Java Filename: B.java

```
javac B.java
```

```
java B
```

8. Output Screen

the value of x is :10

the value of y is :20

9. Result

The program was successfully implemented and executed. The experiment demonstrates **Single-Level Inheritance**, where class B inherits the method of class A using the extends keyword.

MULTILEVEL INHERITANCE

1. Basic Information

Experiment No.: 02

Date: 17/09/2026

Title: Implementation of Multilevel Inheritance in Java

2. Objective / Aim

To design and implement a Java program to demonstrate **Multilevel Inheritance**, where a class inherits from another derived class, forming a chain of inheritance.

3. System Requirements

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Hard Disk: Minimum 10 GB free space
- Keyboard and Monitor

Software Requirements

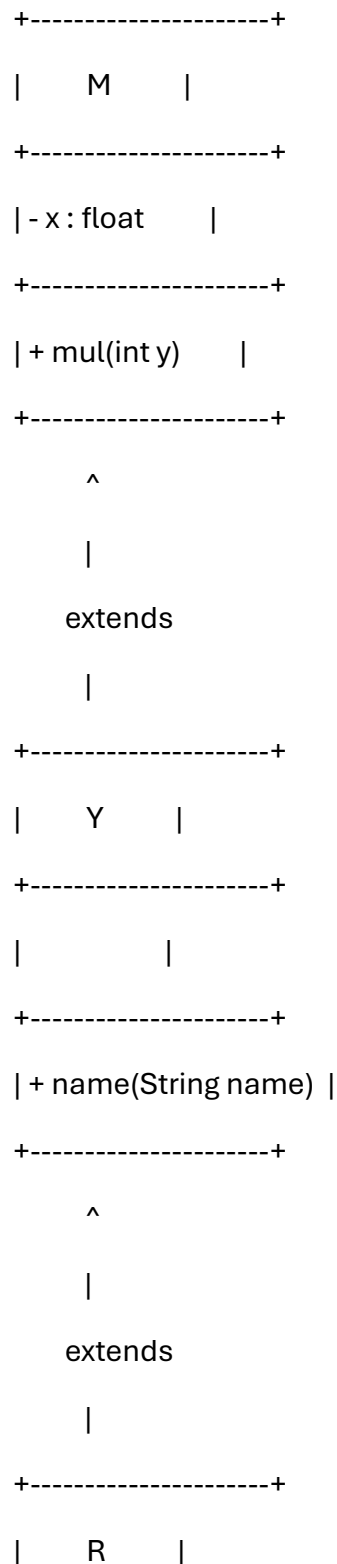
- Operating System: Windows/Linux/macOS
- JDK: JDK 8 or above
- IDE: IntelliJ IDEA, Eclipse, NetBeans, or Visual Studio Code
- Command Prompt/Terminal

4. Algorithm / Logic Description

1. Start the program.
2. Create the superclass M.
3. Declare the variable x in class M.
4. Define the mul() method in class M.
5. Create class Y that extends class M.
6. Define the name() method in class Y.
7. Create class R that extends class Y.
8. Define the div() method in class R.
9. Create an object man of class R.

10. Call mul(), name(), and div() using the object.
11. Display the calculated values and inheritance message.
12. Stop the program.

5. UML Class Diagram



```

+-----+
|         |
+-----+
| + div()   |
+-----+

```

6. Source Code

```

class M{

    float x=98.43f;

    void mul(int y) {

        System.out.println("the mul is :" + (x * y));

    }

}

class Y extends M{

    void name(String name){

        System.out.println("the student "+name+ " secured "+x+" percentage");

        System.out.println(x);

    }

}

class R extends Y{

    void div(){

        System.out.println("this is multilevel inheritance");

    }

}

public class MultiLevel {

    public static void main(String[] args) {

        R man = new R();
    }

}

```

```
man.mul(2);  
man.name("manodhara");  
man.div();  
  
}  
}
```

7. Compilation & Execution Commands

Java Filename: MultiLevel.java

```
javac MultiLevel.java
```

```
java MultiLevel
```

8. Output Screen

the mul is :196.86

the student manodhara secured 98.43 percentage

98.43

this is multilevel inheritance

9. Result

The program was successfully implemented and executed. The experiment demonstrates **Multilevel Inheritance** using the inheritance chain $M \rightarrow Y \rightarrow R$.

HIERARCHICAL INHERITANCE

1. Basic Information

Experiment No.: 03

Date: 17/09/2026

Title: Implementation of Hierarchical Inheritance in Java

2. Objective / Aim

To design and implement a Java program to demonstrate **Hierarchical Inheritance**, where multiple subclasses inherit properties and methods from the same superclass.

3. System Requirements

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Hard Disk: Minimum 10 GB free space
- Keyboard and Monitor

Software Requirements

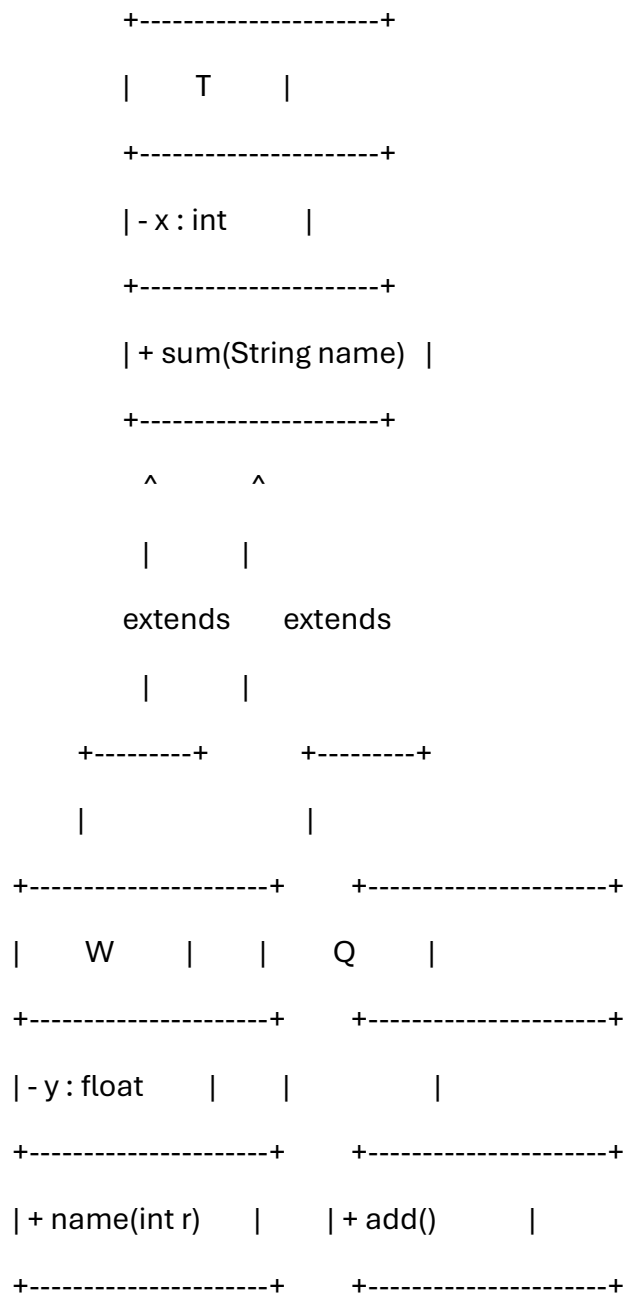
- Operating System: Windows/Linux/macOS
- JDK: JDK 8 or above
- IDE: IntelliJ IDEA, Eclipse, NetBeans, or Visual Studio Code
- Command Prompt/Terminal

4. Algorithm / Logic Description

1. Start the program.
2. Create a superclass T.
3. Declare the variable x in class T.
4. Define the sum() method in class T.
5. Create subclass W that extends class T.
6. Declare variable y and define the name() method in class W.
7. Create another subclass Q that also extends class T.
8. Define the add() method in class Q.
9. Create objects q and w for classes Q and W.

10. Call the add() method using object q.
11. Call the name() method using object w.
12. Call the inherited sum() method using object q.
13. Display the output.
14. Stop the program.

5. UML Class Diagram



6. Source Code

```
class T{
```

```

int x;

void sum(String name){

    System.out.println("the name of person is :"+name);

}

}

class W extends T{

    float y=23.45f;

    void name(int r){

        System.out.println("the value of r is:"+r);

    }

}

class Q extends T{

    void add(){

        System.out.println("this is heirarichal inheritance");

    }

}

public class Heirarichal {

    public static void main(String[] args) {

        Q q=new Q();

        W w=new W();

        q.add();

        w.name(34);

        q.sum("manodhara");

    }

}

```

7. Compilation & Execution Commands

Java Filename: Heirarichal.java

```
javac Heirarichal.java
```

```
java Heirarichal
```

8. Output Screen

this is heirarichal inheritance

the value of r is:34

the name of person is :manodhara

9. Result

The program was successfully implemented and executed. The experiment demonstrates **Hierarchical Inheritance**, where classes W and Q inherit from the same superclass T.

HYBRID INHERITANCE

1. Basic Information

Experiment No.: 04

Date: 17/09/2026

Title: Implementation of Hybrid Inheritance in Java

2. Objective / Aim

To design and implement a Java program to demonstrate **Hybrid Inheritance** using a combination of multilevel and hierarchical inheritance.

3. System Requirements

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Hard Disk: Minimum 10 GB free space
- Keyboard and Monitor

Software Requirements

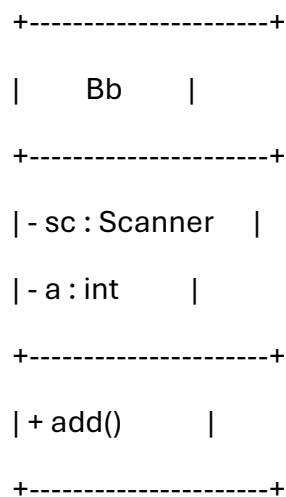
- Operating System: Windows/Linux/macOS
- JDK: JDK 8 or above

- IDE: IntelliJ IDEA, Eclipse, NetBeans, or Visual Studio Code
- Command Prompt/Terminal

4. Algorithm / Logic Description

1. Start the program.
2. Import the Scanner class to read input from the user.
3. Create superclass Bb.
4. Declare integer variable a and define the add() method.
5. Read and display the value of a.
6. Create class Ss that extends Bb.
7. Declare integer variable b and define the sum() method.
8. Read and display the value of b.
9. Calculate and display the sum of a and b.
10. Create class Ww that extends Ss and define the eds() method.
11. Create class Rr that also extends Ss and define the sub() method.
12. Create objects re and ew of classes Ww and Rr.
13. Use object re to call inherited methods add() and sum(), followed by eds().
14. Use object ew to call inherited methods add() and sum(), followed by sub().
15. Display the results.
16. Stop the program.

5. UML Class Diagram




```
Scanner sc=new Scanner(System.in);

int a;

void add(){

    System.out.println("enter the value of a :");

    a=sc.nextInt();

    System.out.println("the value of a is :"+a);

    System.out.println("this is a method in super class ");

}

}

class Ss extends Bb{

int b;

void sum(){

    System.out.println("enter the value of b :");

    b=sc.nextInt();

    System.out.println("the value of b is :"+b);

    System.out.println("the sum of a and b is :"+(a+b));

}

}

class Ww extends Ss{

void eds(){

    System.out.println("this is hybrid inheritance");

}

}

class Rr extends Ss{

void sub(){

    System.out.println("the diff between a and b is :"+(a-b));

}

}
```

```
public class HybridInheritance {  
    public static void main(String[] args) {  
        Ww re=new Ww();  
        Rr ew=new Rr();  
        re.add();  
        re.sum();  
        re.eds();  
        ew.add();  
        ew.sum();  
        ew.sub();  
    }  
}
```

7. Compilation & Execution Commands

Java Filename: HybridInheritance.java

```
javac HybridInheritance.java
```

```
java HybridInheritance
```

8. Output Screen

Sample Input/Output:

enter the value of a :

10

the value of a is :10

this is a method in super class

enter the value of b :

20

the value of b is :20

the sum of a and b is :30

this is hybrid inheritance

enter the value of a :

30

the value of a is :30

this is a method in super class

enter the value of b :

10

the value of b is :10

the sum of a and b is :40

the diff between a and b is :20

9. Result

The program was successfully implemented and executed. The experiment demonstrates **Hybrid Inheritance** by combining multilevel and hierarchical inheritance using classes Bb, Ss, Ww, and Rr.

OVERALL RESULT

The Java programs for different **types of inheritance** were successfully implemented and executed. The experiments demonstrate:

- **Single-Level Inheritance** – One subclass inherits from one superclass.
- **Multilevel Inheritance** – Inheritance occurs through multiple levels.
- **Hierarchical Inheritance** – Multiple subclasses inherit from the same superclass.
- **Hybrid Inheritance** – A combination of different inheritance structures is demonstrated.

The programs use the Java extends keyword to establish inheritance relationships and demonstrate code reusability through inherited methods.